ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Toàn Thắng
Hồ Minh Tiến
Dương Tùng Thiện
Đặng Thái Tú
(Nhóm sinh viên thực hiện)

THỰC HÀNH TRÍ TUỆ NHÂN TẠO NÂNG CAO
LAB 02: GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM
BFS - DFS - DLS - IDS - GBFS - A* - WEIGHTED A*

MÔN HỌC: TRÍ TUỆ NHÂN TẠO NÂNG CAO
NGÀNH: CÔNG NGHỆ THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN:
TS. Đỗ Như Tài

Thành phố Hồ Chí Minh, năm 2026

Mục lục

1. MỤC TIÊU....3
2. BÀI TẬP THỰC HÀNH....4
2.1. Bài 1. Cấu trúc cây tìm kiếm và lớp Node....4
2.2. Bài 2. Mô hình hóa bài toán mê cung....4
2.3. Bài 3. Tìm kiếm không thông tin: BFS và DFS....6
2.4. Bài 4. Depth-Limited DFS và Iterative Deepening Search....9
2.5. Bài 5. Tìm kiếm có thông tin: GBFS, A* và Weighted A*....11
2.6. Bài 6. Ảnh hưởng của heuristic....13
2.7. Bài 7. Kiểm tra chu trình trong DFS....15
2.8. Bài 8. Tổng hợp - Goal-based Planning Agent....15
2.9. Câu hỏi thảo luận....17
3. NHẬN XÉT VÀ KẾT LUẬN....20
4. TÀI LIỆU SỬ DỤNG TRONG BÀI THỰC HÀNH....21
5. DANH MỤC TÀI LIỆU THAM KHẢO....21
PHỤ LỤC A. KẾT QUẢ ĐẦY ĐỦ CỦA WEIGHTED A* (54 LƯỢT CHẠY)....22

Bảng phân công

Họ tên / MSSV	Công việc	Đánh giá mức độ hoàn thành (Trưởng nhóm đánh giá)	Khối lượng (%)
[Họ tên TV1 – MSSV]	- BFS, DFS (Bài 3) - Câu thảo luận 6, 8 - Notebook Ex1: Task 1–2, 4	Hoàn thành phần BFS/DFS; [trưởng nhóm đánh giá]	25%
Dương Tùng Thiện 3123410350	- DLS, IDS (Bài 4) - Câu thảo luận 7, 12	Hoàn thành, đã kiểm chứng lại số liệu Small; [trưởng nhóm đánh giá]	25%
[Họ tên TV3 – MSSV]	- GBFS, A* (Task 3 của Ex1, Bài 5) - Câu thảo luận 9, 10	[Chưa hoàn thành – chờ bổ sung]	25%
[Họ tên TV4 – MSSV]	- Weighted A*, heuristic Obstacle-Aware (Bài 5, 6) - Câu thảo luận 6, 11	Hoàn thành 54 lượt chạy; [trưởng nhóm đánh giá]	25%
Nguyễn Toàn Thắng (Nhóm trưởng) [xác nhận]	- Tổng hợp, viết báo cáo chung	[trưởng nhóm đánh giá]	—

Ghi chú: các mục tô vàng là thông tin cần nhóm điền hoặc xác nhận trước khi nộp.

1. MỤC TIÊU

- Hiểu cách mô hình hóa bài toán tìm kiếm bằng không gian trạng thái, trạng thái đầu, trạng thái đích, hành động, chi phí đường đi và cây tìm kiếm.
- Cài đặt và khảo sát các chiến lược tìm kiếm không thông tin: BFS, DFS, Depth-Limited DFS (DLS) và Iterative Deepening Search (IDS).
- Cài đặt và khảo sát các chiến lược tìm kiếm có thông tin: Greedy Best-First Search (GBFS), A* và Weighted A*.
- Hiểu vai trò của heuristic Manhattan/Euclidean và ảnh hưởng của kiểm tra chu trình (cycle checking).
- Đánh giá thuật toán theo độ dài đường đi, số nút mở rộng, độ sâu, kích thước frontier và thời gian chạy.

2. BÀI TẬP THỰC HÀNH

2.1. Bài 1. Cấu trúc cây tìm kiếm và lớp Node

Cây tìm kiếm trong AI khác với cây nhị phân trong môn Cấu trúc dữ liệu: cây được **tạo động trong lúc tìm kiếm**, mỗi nút có thể có nhiều con (mỗi con ứng với một hành động) và các nhánh không còn cần thiết có thể bị giải phóng để tiết kiệm bộ nhớ (ví dụ trong DFS). Vì vậy, nút con giữ tham chiếu đến nút cha (parent) thay vì nút cha giữ danh sách con: khi đi xuống cây ta luôn đang tạo nút mới nên đã có sẵn tham chiếu, còn liên kết lên chỉ cần dùng để truy hồi đường đi.

Nhóm sử dụng lớp Node trong 01_trees.ipynb với bốn thuộc tính: pos (trạng thái, là cặp (hàng, cột)), parent (nút cha, None nếu là nút gốc), action (hành động dẫn từ cha đến nút này, None ở gốc) và cost (chi phí tích lũy g(n); với chi phí bước bằng 1 thì cost bằng độ sâu).

```
class Node:
    def __init__(self, pos, parent, action, cost):
        self.pos = tuple(pos)    # trạng thái (row, col)
        self.parent = parent     # nút cha, None = nút gôć
        self.action = action     # hành động sinh ra nút này
        self.cost = cost         # g(n)

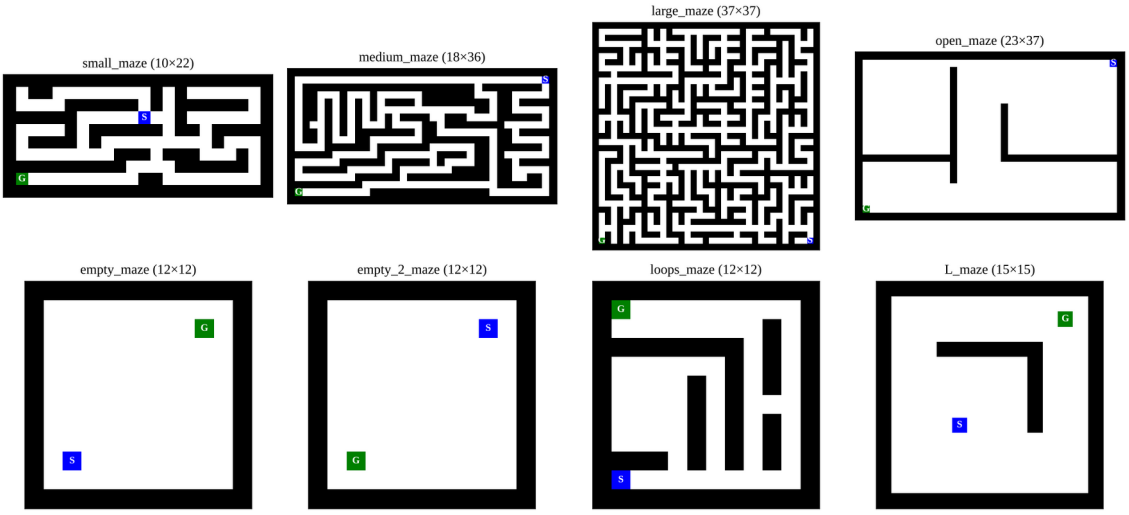
    def get_path_from_root(self):
        node, path = self, [self]
        while node.parent is not None:
            node = node.parent; path.append(node)
        path.reverse(); return path
```

Khi chạy notebook: nút gốc Node((0,0), None, None, 0) ban đầu có reference count bằng 1; sau khi tạo node2 = Node((0,1), root, 'E', 1), nút gốc được tham chiếu hai lần (biến root và node2.parent). Ngay cả khi gán root = None, nút gốc vẫn tồn tại vì node2.parent còn trỏ tới nó; chỉ khi bỏ node2 thì toàn bộ cây mới bị garbage collector thu hồi. Gọi node2.get_path_from_root() trả về vị trí [(0,0), (0,1)], hành động [None, 'E'] và chi phí [0, 1]. Đây chính là cách các thuật toán trong báo cáo truy hồi lời giải từ nút đích về gốc.

2.2. Bài 2. Mô hình hóa bài toán mê cung

2.2.1. Năm thành phần của bài toán tìm kiếm

Các mê cung được lưu trong file văn bản: X là tường, S là điểm bắt đầu, G là đích, ô trống là ô có thể đi. Hình 1 là toàn bộ tám mê cung dùng trong bài (kết quả của Ex3_Show_all_mazes.ipynb).



Hình 1. Tám mê cung dùng trong bài thực hành (S: xanh dương, G: xanh lá, tường: đen).

Thành phần	Định nghĩa cho bài toán mê cung
Không gian trạng thái	Mê cung là bảng R × C. Một trạng thái là vị trí s = (r, c) của tác tử trên một ô không phải tường.
Trạng thái đầu	Vị trí ô S: s_0 = (r_S, c_S).
Hành động	ACTIONS(s) = { a ∈ {N, E, S, W} | s + Δ(a) không là tường }, với Δ(N) = (−1, 0), Δ(E) = (0, +1), Δ(S) = (+1, 0), Δ(W) = (0, −1).
Mô hình chuyển trạng thái	Tất định: RESULT(s, a) = s + Δ(a) = (r + Δr, c + Δc).
Trạng thái đích	Ô G: GOAL-TEST(s) = true nếu s = (r_G, c_G).
Chi phí đường đi	Mỗi bước có chi phí c(s, a, s') = 1, nên g(n) bằng số bước từ s_0 đến n. Lời giải tối ưu là chuỗi hành động ít bước nhất.

Bảng 1. Năm thành phần của bài toán tìm kiếm trong mê cung.

Môi trường của tác tử là quan sát đầy đủ (tác tử luôn biết tọa độ), rời rạc (hữu hạn ô), tất định và đã biết (tác tử có bản đồ). Khi đó tác tử lập kế hoạch một lần rồi thực hiện tuần tự (open-loop), không cần quan tâm đến percept trong lúc thực thi.

2.2.2. Ước lượng n, d, m, b

n là kích thước không gian trạng thái (số ô không phải tường; giá trị chính xác là số ô đến được từ S, tính bằng BFS); d là độ sâu của lời giải tối ưu (path cost của BFS vì mỗi bước có chi phí 1); m là độ sâu tối đa của cây tìm kiếm (với BFS xấp xỉ khoảng cách xa nhất từ S; với DFS chỉ kiểm tra chu trình trên đường đi thì m ≤ n − 1); b là hệ số phân nhánh tối đa, đếm số lân cận không phải tường của từng ô rồi lấy giá trị lớn

nhất ($b \leq 4$, và nút không phải gốc có tối đa $b - 1$ con mới vì một hướng quay lại nút cha). Bảng 2 là kết quả đo trên tám mê cung.

Mê cung	Kích thước	n (số ô đi được)	d	m (BFS, khoảng cách xa nhất)	b
small	10×22	94	19	20	3
medium	18×36	274	68	70	3
large	37×37	647	210	222	4
open	23×37	684	54	54	4
empty	12×12	100	14	16	4
empty_2	12×12	100	14	16	4
loops	12×12	72	23	23	4
L	15×15	157	16	18	4

Bảng 2. Ước lượng kích thước bài toán trên các mê cung (đo bằng BFS thuần để phân tích).

2.3. Bài 3. Tìm kiếm không thông tin: BFS và DFS

2.3.1. Cài đặt

Cả hai thuật toán tái sử dụng lớp Node và hàm expand(maze, node) (sinh các nút con không đi vào tường) theo thứ tự hành động N, E, S, W. Thông tin tìm kiếm chỉ được lưu trong cây tìm kiếm, frontier và reached, **không ghi vào bản đồ mê cung**. Cả hai kiểm tra đích ngay khi sinh nút con (early goal test) như pseudocode BREADTH-FIRST-SEARCH của sách AIMA; vì mọi bước có chi phí 1 nên BFS vẫn tối ưu và nút đích không phải đi qua frontier.

- **BFS** dùng hàng đợi FIFO (collections.deque) và tập reached; một ô chỉ được đưa vào frontier nếu chưa từng được gặp, nên mỗi trạng thái vào frontier tối đa một lần.
- **DFS** dùng ngăn xếp LIFO và **không có reached**. Để tránh vòng lặp, hàm is_cycle(node) lần theo parent về gốc và loại các nút trùng trạng thái với tổ tiên. Nhờ không lưu reached, bộ nhớ chủ yếu là đường đi hiện tại cộng các nút chưa xét trong stack.

```
frontier = deque([root]); reached = {start}          # BFS
while frontier:
    node = frontier.popleft()                         # FIFO
    for child in expand(maze, node):
        if child.pos == goal: return child            # goal test khi sinh
nút con
        if child.pos not in reached:
            reached.add(child.pos); frontier.append(child)

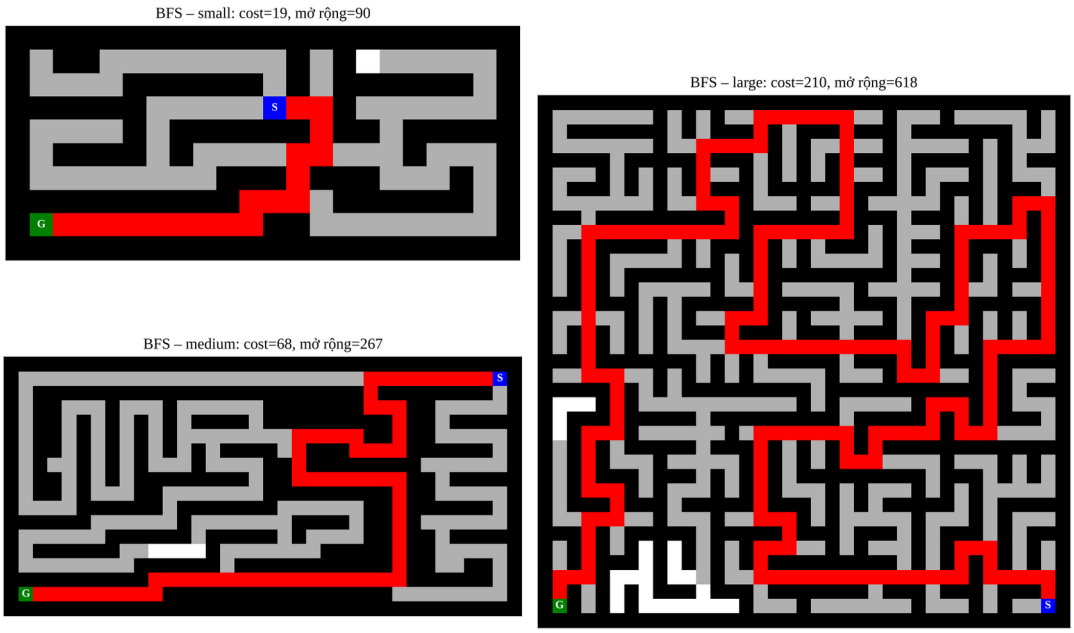
frontier = [root]                                     # DFS
```

```
while frontier:
    node = frontier.pop()                                   # LIFO
    for child in expand(maze, node):
        if is_cycle(child): continue                        # chỉ loại chu trình
trên nhánh
        if child.pos == goal: return child
        frontier.append(child)
```

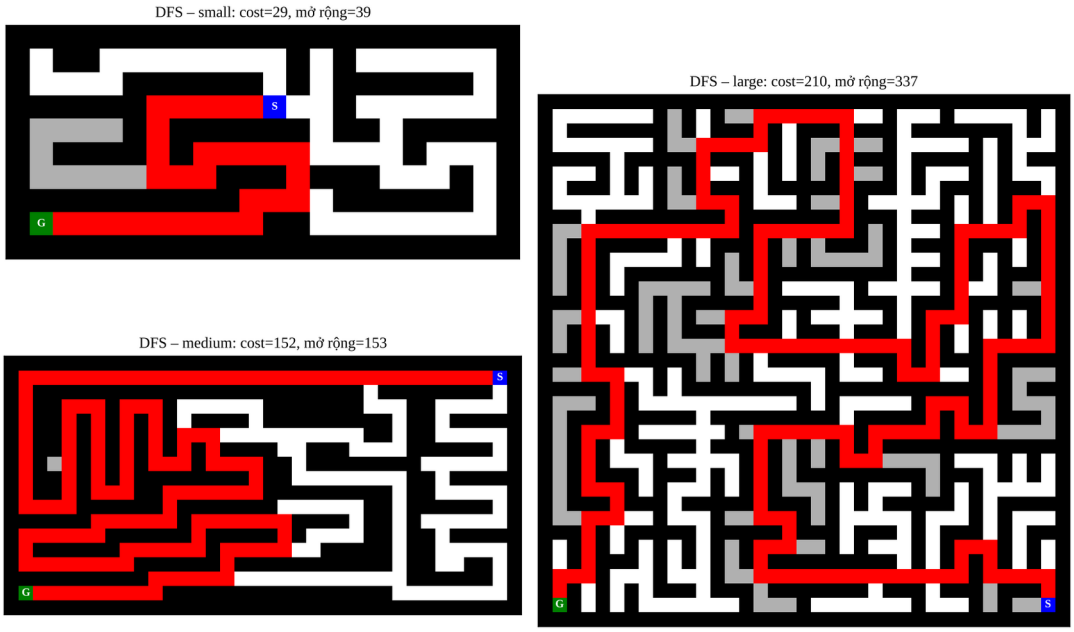
2.3.2. Kết quả trên small, medium, large

Mê cung	Thuật toán	Path cost	Nút mở rộng	Max depth	Max frontier	Thời gian (ms)
small	BFS	19	90	19	9	0.48
small	DFS	29	39	29	5	0.22
medium	BFS	68	267	68	9	1.33
medium	DFS	152	153	152	8	1.78
large	BFS	210	618	210	8	3.08
large	DFS	210	337	214	38	4.94

Bảng 3. So sánh BFS và DFS (thứ tự N, E, S, W). Thời gian là giá trị nhỏ nhất trong 5 lần chạy.



Hình 2. Đường đi của BFS (đỏ) và các ô đã gặp (xám).



Hình 3. Đường đi của DFS (đỏ) và các ô đã mở rộng (xám).

BFS luôn cho path cost nhỏ nhất (19, 68, 210). DFS cho đường dài hơn nhiều ở small (29) và medium (152) và chỉ bằng BFS ở large (210) – nhưng max depth của DFS ở large là 214 > 210 vì nó từng đi vào nhánh sâu hơn đường cuối cùng. DFS mở rộng ít nút hơn BFS ở cả ba mê cung (39/90, 153/267, 337/618) vì đi thẳng một nhánh, song ít nút hơn không có nghĩa là lời giải tốt hơn. Về frontier, DFS nhỏ hơn BFS ở small và medium nhưng lớn hơn ở large (38 so với 8); cột này chỉ đo frontier, còn BFS giữ thêm tập reached, nên không thể kết luận DFS luôn tốn ít bộ nhớ hơn chỉ từ cột này. Thời gian chỉ ở mức vài mili-giây, khá nhiễu nên chỉ dùng tham khảo.

2.3.3. Queue và Stack; tính đầy đủ, tối ưu, độ phức tạp

Tiêu chí	BFS (Queue – FIFO)	DFS (Stack – LIFO)
Thứ tự lấy nút	Vào trước, ra trước: duyệt hết độ sâu d rồi mới sang d+1	Vào sau, ra trước: đi sâu hết một nhánh rồi mới quay lại
Tối ưu (chi phí bước bằng nhau)	Có – nghiệm đầu tiên là nông nhất	Không đảm bảo
Đầy đủ trong không gian hữu hạn	Có	Có nếu kiểm tra chu trình đúng; nếu không có thể lặp vô hạn
Thời gian	O(n) với reached (tổng quát O(b^d))	Tệ nhất O(b^m): cùng một ô bị mở rộng lại qua nhiều đường
Bộ nhớ	O(n) – giữ frontier và reached	O(b·m) – chỉ đường đi hiện tại và các nút anh em

Bảng 4. So sánh BFS và DFS về frontier, tính chất và độ phức tạp.

Kiểm tra chu trình của DFS chỉ phát hiện vòng lặp **trên cùng một đường đi**; nó không loại được việc đến cùng một ô qua nhiều đường khác nhau. Hệ quả thấy rõ ở các mê cung mở: khi chạy DFS cài đặt này trên open_maze, chương trình không kết thúc trong 20 giây (số đường đi không lặp ô tăng theo cấp số nhân), trong khi BFS giải xong với 679 nút mở rộng. Vì vậy cần các cơ chế cycle checking mạnh hơn (xem Bài 7).

2.4. Bài 4. Depth-Limited DFS và Iterative Deepening Search

2.4.1. Nguyên lý và cài đặt

DLS là DFS dùng stack với giới hạn độ sâu limit. Khi lấy một nút khỏi stack, thuật toán kiểm tra đích trước; nếu nút đã ở độ sâu limit thì không mở rộng, và chỉ đánh dấu cutoff khi còn ít nhất một nút con hợp lệ (không tạo chu trình) nằm ngoài giới hạn. DLS trả về một trong ba trạng thái: found, cutoff hoặc failure. Chu trình được kiểm tra bằng is_cycle() trên chuỗi nút cha.

IDS gọi DLS lần lượt với limit = 0, 1, 2, … cho đến khi found; nếu DLS trả về failure (không còn nhánh nào bị cắt) thì kết luận không có lời giải, còn cutoff thì tăng limit. Với chi phí bước bằng 1, giới hạn đầu tiên tìm thấy đích cũng cho đường đi ngắn nhất, nên IDS vừa có bộ nhớ nhỏ của DFS vừa có tính đầy đủ và tối ưu của BFS. Với IDS, số nút mở rộng được cộng qua mọi vòng DLS, còn max frontier là giá trị lớn nhất của từng vòng (không phải tổng các stack).

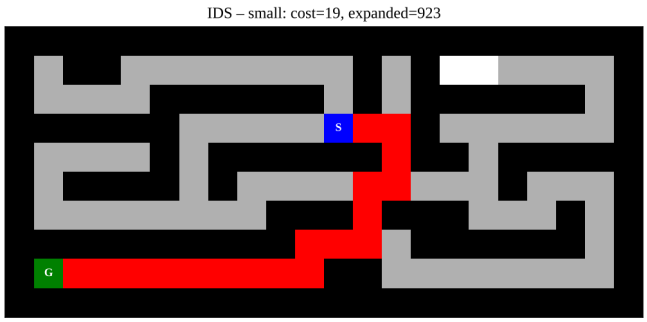
2.4.2. Kết quả

Giới hạn độ sâu	Trạng thái	Path cost	Nút mở rộng
0	cutoff	—	0
18	cutoff	—	114
19	found	19	72
20	found	19	79

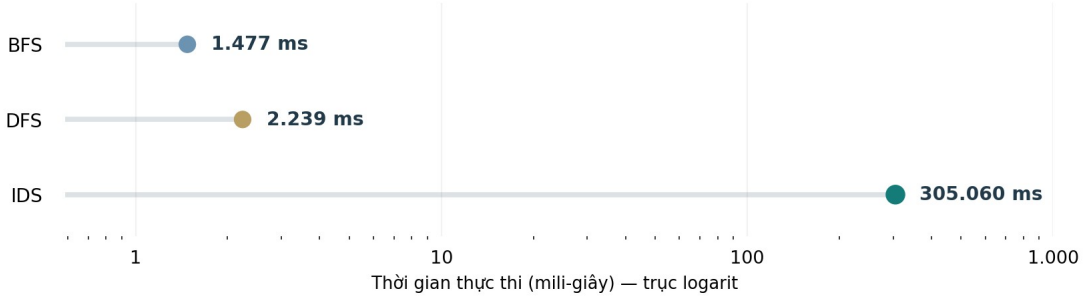
Bảng 5. Kiểm tra DLS trên small maze với các giới hạn khác nhau.

Mê cung	Path cost	Số vòng DLS	Số lần tăng limit	Nút mở rộng	Max depth	Max frontier	Thời gian (ms)
small	19	20	19	923	19	6	5.53
medium	68	69	68	15,665	68	8	67.13
large	210	211	210	60,179	210	38	307.27

Bảng 6. Kết quả IDS (từ notebook của TV2). Với limit = 18 DLS chưa đến được G; từ limit = 19 thì tìm thấy, khớp với 20 lần gọi DLS (limit 0..19) của IDS.



Hình 4. IDS trên small maze: đường đi cost = 19 nối S (xanh dương) và G (xanh lá).



Hình 5. Thời gian chạy IDS so với BFS và DFS trên large maze (trục logarit).

2.4.3. Nhận xét

- **Chất lượng lời giải:** IDS cho cost 19, 68, 210, trùng BFS – phù hợp với tính tối ưu theo số bước.
- **Nút mở rộng:** các nút nông bị mở rộng lại ở mỗi vòng nên trên large IDS mở rộng 60,179 nút so với 618 của BFS và 337 của DFS.
- **Frontier và bộ nhớ:** max frontier của IDS trên large là 38 (BFS là 8). Chỉ số này không phản ánh toàn bộ bộ nhớ: code còn lưu Node/parent và tập visited dùng để vẽ hình. Nếu cần khảo sát chặt chẽ nên tắt lưu visited khi benchmark và đo peak memory thực tế.
- **Thời gian:** IDS chậm hơn BFS/DFS hàng trăm lần trên large (305 ms so với 1.5–2.2 ms). IDS cũng phụ thuộc cơ chế cycle checking của DFS nên không phù hợp với các mê cung mở như open hoặc empty.
- **Kiểm chứng:** TV2 chạy lại IDS trên small, thu được cost 19, 923 nút mở rộng, max frontier 6; kiểm tra thêm 75 mê cung ngẫu nhiên nhỏ thấy kết quả thống nhất với BFS tham chiếu (44 mê cung có đường đi, 31 không có). Khi biên soạn báo cáo chung, chúng tôi chạy lại IDS trên cả ba mê cung và nhận được cùng số nút mở rộng, max depth và max frontier.

2.5. Bài 5. Tìm kiếm có thông tin: GBFS, A* và Weighted A*

2.5.1. Greedy Best-First Search và A (phần của TV3)*

[PHẦN CỦA THÀNH VIÊN 3 – CHƯA HOÀN THÀNH. Cần bổ sung: mô tả cài đặt GBFS (f(n) = h(n)) và A* (f(n) = g(n) + h(n)) với h = Manhattan trong Ex1_Maze.ipynb Task 3; kết quả trên small, medium, large; hình đường đi; thảo luận tính đầy đủ/tối ưu và độ phức tạp.]

Mê cung	Thuật toán	Path cost	Nút mở rộng	Max depth	Max frontier	Thời gian (ms)
small	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
medium	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
large	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
small	A*	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
medium	A*	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
large	A*	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]

Bảng 7. Bảng kết quả GBFS và A của TV3 (chờ bổ sung).*

Số liệu tham chiếu tạm thời: A* với heuristic Manhattan chính là Weighted A* với w = 1 trong phần của TV4 (Mục 2.5.3), cho path cost 19 / 68 / 210 và số nút mở rộng 53 / 221 / 549 trên small / medium / large – bằng BFS về chất lượng lời giải nhưng mở rộng ít nút hơn (BFS: 90 / 267 / 618).

2.5.2. So sánh BFS và A (04_Maze_BFS_vs_A_Star.ipynb)*

Notebook 04 so sánh trực tiếp BFS và A* trên L_maze: cả hai đều tối ưu với path length 16, nhưng BFS gặp 151 ô (gần như toàn bộ mê cung) còn A* chỉ gặp 87 ô với heuristic Euclidean và 66 ô với heuristic Manhattan. BFS là thuật toán không thông tin nên lan ra theo các vòng tròn đồng tâm quanh S; A* được heuristic dẫn về phía G nên cây tìm kiếm nhỏ hơn. Hiệu quả của A* phụ thuộc chất lượng heuristic – Manhattan rất tốt cho lưới 4 hướng. Khi cây của A* vẫn quá lớn so với bộ nhớ, có thể dùng Weighted A* hoặc IDS.

Thuật toán (L_maze)	Path length	Số ô đã gặp
BFS	16	151
A* – Euclidean	16	87
A* – Manhattan	16	66

Bảng 8. BFS và A trên L_maze (số liệu từ output notebook 04, cài đặt của giảng viên).*

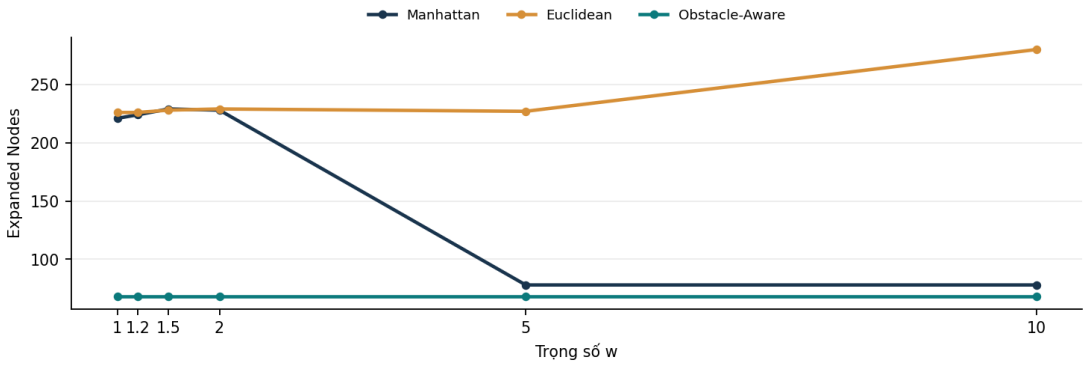
2.5.3. Weighted A (phần của TV4)*

Weighted A* dùng f(n) = g(n) + w × h(n) với w ≥ 1; w = 1 là A* chuẩn. Frontier là hàng đợi ưu tiên (heapq); chỉ h(n) được nhân trọng số, không phải toàn bộ f(n). Node và expand() được tái sử dụng từ Ex1. Thực nghiệm gồm 54 lượt chạy: ba mê cung (small, medium, large) × sáu trọng số w ∈ {1; 1.2; 1.5; 2; 5; 10} × ba heuristic (Manhattan, Euclidean, Obstacle-Aware – xem Bài 6). Các chỉ số: Path Cost, Expanded, Max Depth (giá trị g lớn nhất của nút được đẩy vào frontier), Max Frontier (số phần tử cực đại trong heapq). Thời gian chỉ là một lần chạy nên ưu tiên so sánh Path Cost, Expanded và Frontier.

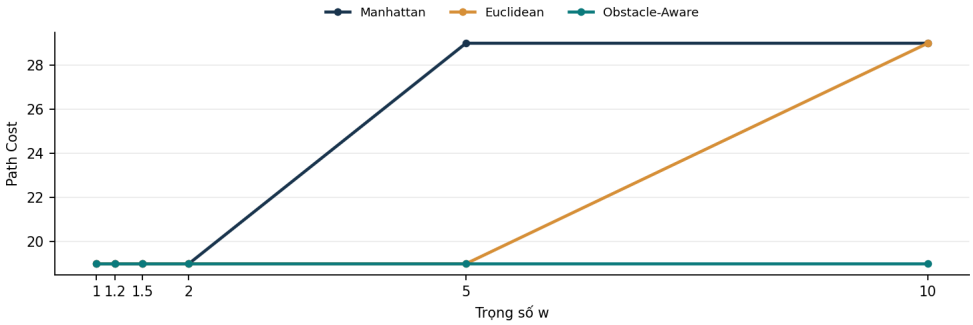
Mê cung	Heuristic	w = 1: C / E / D / F	w = 10: C / E / D / F
Small	Manhattan	19 / 53 / 19 / 8	29 / 39 / 29 / 5
Small	Euclidean	19 / 56 / 19 / 7	29 / 54 / 29 / 4
Small	Obstacle-Aware	19 / 19 / 19 / 7	19 / 19 / 19 / 7
Medium	Manhattan	68 / 221 / 68 / 8	74 / 78 / 74 / 4
Medium	Euclidean	68 / 226 / 68 / 8	102 / 280 / 128 / 10
Medium	Obstacle-Aware	68 / 68 / 68 / 7	68 / 68 / 68 / 7
Large	Manhattan	210 / 549 / 210 / 12	210 / 475 / 210 / 21
Large	Euclidean	210 / 557 / 210 / 10	210 / 488 / 210 / 18
Large	Obstacle-Aware	210 / 210 / 210 / 67	210 / 210 / 210 / 67

Bảng 9. Weighted A ở hai đầu khoảng trọng số. C / E / D / F = Path Cost / Expanded / Max Depth / Max Frontier (đầy đủ 54 lượt chạy ở Phụ lục A).*

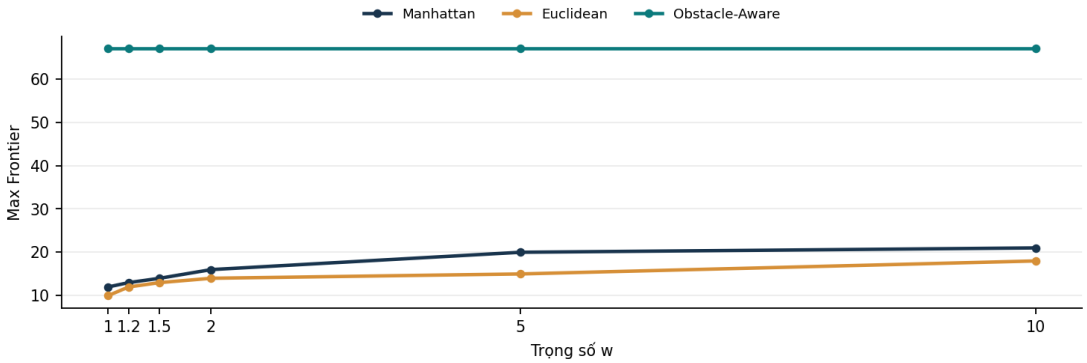
Small. Manhattan khi tăng w từ 1 lên 10 giảm Expanded từ 53 xuống 39 (−26.4%) và Max Frontier từ 8 xuống 5, nhưng Path Cost tăng từ 19 lên 29 (+52.6%). **Medium.** Manhattan ở w = 5 giảm Expanded từ 221 còn 78 (−64.7%), Cost tăng từ 68 lên 74 (+8.8%); ngược lại Euclidean ở w = 10 tăng Expanded từ 226 lên 280, Cost từ 68 lên 102 – trọng số lớn không tự động bảo đảm mở rộng ít nút hơn. **Large.** Manhattan giảm Expanded từ 549 còn 475 (−13.5%), Euclidean từ 557 còn 488 (−12.4%), cả hai giữ Cost = 210; tuy vậy Max Frontier tăng từ 12 lên 21 (+75%) và từ 10 lên 18 (+80%), tức giảm số nút mở rộng không đồng nghĩa giảm nhu cầu bộ nhớ.



Hình 6. Medium maze: số nút mở rộng thay đổi theo w (ba heuristic).



Hình 7. Small maze: Path Cost tăng ở trọng số lớn dù Expanded có thể giảm.



Hình 8. Large maze: kích thước frontier cực đại theo w.

2.6. Bài 6. Ảnh hưởng của heuristic

2.6.1. Khảo sát trên mê cung rỗng (06_Explore_heuristics.ipynb)

Notebook 06 dùng mê cung rỗng 10 × 10 với S = (1, 1), G = (8, 8) để quan sát giá trị h(n) và f(n):

- **Manhattan** h(n) = |x_n − x_g| + |y_n − y_g| là heuristic hoàn hảo trong lưới 4 hướng không vật cản: h(n) = h*(n). Tuy vậy mọi đường đi trực tiếp từ S đến G đều có cùng độ dài, nên A* với Manhattan gặp một **vùng phẳng** (plateau) các ô có cùng f(n) = C*; nếu dùng bảng reached thì toàn bộ vùng này bị mở rộng và tìm kiếm kém hiệu quả. Cách tiêu biểu là phá hòa (tie-breaking) hoặc tăng nhẹ h(n).

- **Weighted A*:** w = 1.1 tạo gradient nhỏ dẫn thẳng từ S đến G; w = 5 tiến gần GBFS (gần như bỏ qua g(n)); w < 1 làm thuật toán giống tìm kiếm chi phí đồng nhất/BFS, lan ra quanh S và rất kém hiệu quả.
- **Euclidean** $h(n) = \sqrt{((x_n - x_g)^2 + (y_n - y_g)^2)}$ là admissible nhưng không hoàn hảo; với g(n) tính theo Manhattan thì gradient của f(n) = g(n) + h(n) đi từ G về phía S, nên A* mở rộng mọi nút có f(n) < C* và một số nút có f(n) = C*.

Kết luận của notebook: cần chọn heuristic tạo ra gradient rõ ràng cho A* bám theo, nếu không thuật toán sẽ kém hiệu quả như BFS.

2.6.2. Heuristic mới: Obstacle-Aware Distance (TV4)

Obstacle-Aware: h(n) = d*(n, Goal), với bảng d* được tính trước bằng một lượt BFS từ Goal trên toàn bộ ô đi được. Vì mê cung đi 4 hướng với chi phí bước bằng 1, đây là heuristic chính xác (perfect heuristic), dùng làm mốc tham chiếu chất lượng; nó không phải phương pháp rẻ để tính cho một truy vấn đơn lẻ vì cần BFS tiền xử lý.

Tại w = 1, Obstacle-Aware mở rộng 19 / 68 / 210 nút trên small / medium / large, giảm khoảng 64.2% / 69.2% / 61.7% so với Manhattan (53 / 221 / 549), và giữ path cost tối ưu ở mọi w đã thử. Đổi lại, trên large Max Frontier của Obstacle-Aware luôn là 67, cao hơn Manhattan (12 ở w = 1; 21 ở w = 10).

Mê cung	Tiền xử lý OA (ms)	OA tìm kiếm, w=1 (ms)	OA tổng (ms)	Manhattan tìm kiếm, w=1 (ms)
Small	0.4267	0.0790	0.5057	0.3299
Medium	0.7662	0.2478	1.0140	1.2673
Large	1.7026	0.9101	2.6127	3.2585

Bảng 10. Chi phí tiền xử lý Obstacle-Aware (một lần chạy; tổng = tìm kiếm + tiền xử lý). Chỉ so Search Time sẽ thiên vị Obstacle-Aware; các số ms chỉ minh họa và cần lặp nhiều lần nếu kết luận về tốc độ.

*2.6.3. Điều kiện để heuristic giữ tính tối ưu của A**

Với bước đi 4 hướng chi phí 1, Manhattan và Euclidean đều admissible ($h(n) \le h^*(n)$) và consistent ($h(n) \le c(n, n') + h(n')$). Obstacle-Aware dùng khoảng cách ngắn nhất thực tế nên cũng admissible và consistent. Khi w = 1 và quản lý reached/frontier đúng cách thì A* tối ưu; với w > 1 bảo đảm tối ưu thông thường không còn. Một heuristic đề xuất bất kỳ chỉ giữ được tối ưu của A* khi không bao giờ đánh giá quá chi phí thật còn lại (admissible) và, nếu dùng bảng reached, phải consistent.

2.7. Bài 7. Kiểm tra chu trình trong DFS

Notebook 05_Maze_Example-Animation_Cycle_checking.ipynb chạy DFS trên open_maze với ba phương án kiểm tra chu trình:

Phương án	Cách làm	Kết quả (open_maze, tối đa 500 lượt)
1	Kiểm tra trạng thái đã có trên đường đi hiện tại	Không tìm ra lời giải. Cùng một trạng thái bị đẩy vào frontier nhiều lần, bị xóa rồi khám phá lại; vùng trống đỏ kín nhưng vẫn còn điểm trong frontier
2	Kiểm tra đường đi + không thêm trạng thái đã có trong frontier	Không tìm ra lời giải: vẫn lặp vô hạn nếu mê cung có vùng trống. Khi DFS xóa một nhánh đã hoàn tất, trong frontier vẫn còn các mục (ô cam) nên sẽ bị khám phá lại
3	Kiểm tra đường đi + đưa trạng thái tìm thấy lại lên đầu stack (xóa mục cũ rồi push lại)	Tìm được lời giải path length = 54 (bằng đường tối ưu của BFS); không còn mục thừa trong frontier khi một nhánh bị xóa khỏi bộ nhớ

Bảng 11. Ba phương án cycle checking cho DFS.

Kết quả cho thấy cycle checking chỉ trên đường đi là chưa đủ trong không gian mở: DFS phải quản lý frontier cẩn thận (phương án 3) để số lần lặp trạng thái và kích thước frontier không bùng nổ. Kết quả này khớp với thử nghiệm của nhóm: DFS ở Ex1 (chỉ kiểm tra tổ tiên) không kết thúc trên open_maze trong 20 giây. Bù lại, phương án 3 và việc duy trì cấu trúc kiểm tra làm tăng chi phí quản lý dữ liệu (xem Câu 12).

2.8. Bài 8. Tổng hợp - Goal-based Planning Agent

Ex1_Maze.ipynb là bài thực hành chính; Ex2_Maze_simplified.ipynb là phiên bản rút gọn chỉ gồm BFS, và Ex3_Show_all_mazes.ipynb hiển thị toàn bộ mê cung (Hình 1). Tất cả thuật toán đều theo mô hình planning agent: hàm lập kế hoạch trả về chuỗi hành động (N, E, S, W) rồi tác tử thực hiện tuần tự. Bảng 12 là bảng ghi kết quả thí nghiệm tổng hợp theo mẫu của đề; kết quả BFS/DFS/A*(w=1) chạy bằng đúng code của nhóm và các giá trị cost, nút mở rộng, max depth, max frontier khớp với số liệu trong báo cáo từng thành viên.

Maze	Thuật toán	Path cost	Nút mở rộng	Max depth	Max frontier	Thời gian (ms)
small	BFS	19	90	19	9	0.48
small	DFS	29	39	29	5	0.22
small	IDS	19	923	19	6	5.29
small	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
small	A* (w=1, TV4)	19	53	19	8	0.44
medium	BFS	68	267	68	9	1.33
medium	DFS	152	153	152	8	1.78
medium	IDS	68	15,665	68	8	98.72
medium	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
medium	A* (w=1, TV4)	68	221	68	8	2.00
large	BFS	210	618	210	8	3.08
large	DFS	210	337	214	38	4.94
large	IDS	210	60,179	210	38	478.88
large	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
large	A* (w=1, TV4)	210	549	210	12	5.00
open	BFS	54	679	54	25	3.88
open	DFS (chỉ kiểm tra tổ tiên)	không kết thúc	> 20 s	—	—	—
open	DFS + cycle check (PA 3, nb 05)	54	—	—	—	—
open	A* (w=1, TV4)	54	535	54	25	3.53
open	GBFS	[TV3]	[TV3]	[TV3]	[TV3]	[TV3]
empty	BFS	14	92	14	12	0.36
empty	DFS	80	80	80	73	0.73
empty	A* (w=1, TV4)	14	63	14	31	0.49
empty_2	BFS	14	92	14	11	0.31
empty_2	DFS	16	16	16	25	0.08
empty_2	A* (w=1, TV4)	14	63	14	31	0.55
loops	BFS	23	70	23	8	0.25
loops	DFS	27	30	27	12	0.13
loops	A* (w=1, TV4)	23	58	23	6	0.37
L	BFS	16	146	16	17	0.48
L	DFS	120	130	120	107	1.34
L	A* (w=1, TV4)	16	53	16	26	0.36

Bảng 12. Bảng ghi kết quả thí nghiệm. A: heuristic Manhattan, goal test khi lấy nút khỏi frontier; GBFS chờ TV3. Thời gian: giá trị nhỏ nhất trong 5 lần chạy (DFS chạy một lần khi mất > 1 ms), phụ thuộc máy chạy.*

Lưu ý khi đọc bảng: BFS/DFS kiểm tra đích khi sinh nút con còn A* kiểm tra khi lấy nút khỏi frontier nên số nút mở rộng không hoàn toàn cùng quy ước; IDS cộng dồn nút mở rộng qua các vòng; trên open_maze không chạy IDS vì cùng hạn chế cycle checking như DFS.

2.9. Câu hỏi thảo luận

Câu 6. Khi nào BFS dùng nhiều bộ nhớ hơn A*? Vì sao heuristic tốt làm giảm vùng trạng thái phải khảo sát?

BFS không có thông tin về đích nên mở rộng mọi nút theo từng lớp độ sâu; khi hệ số nhánh là b và đích ở độ sâu d, frontier có thể chứa tới khoảng b^d nút (tăng theo hàm mũ trong cây, và tới O(n) trong đồ thị mê cung). A* chọn nút theo f(n) = g(n) + h(n), nên heuristic phù hợp ưu tiên các vùng nhiều triển vọng và bỏ qua phần lớn nhánh không hứa hẹn. Vì vậy BFS tốn nhiều bộ nhớ hơn A* khi không gian lớn, nhánh rộng hoặc đích xa, và heuristic của A* đủ tốt.

Ưu thế này phụ thuộc chất lượng heuristic: nếu h(n) quá yếu (xấu nhất h(n) = 0) A* gần như giống BFS/UCS; A* còn phải lưu g, h và con trỏ cha. Số liệu của nhóm minh họa cả hai mặt: A* (w = 1) mở rộng 53 / 221 / 549 nút so với 90 / 267 / 618 của BFS, và trên L_maze A* gặp 66–87 ô so với 151 của BFS; nhưng trên large, Obstacle-Aware mở rộng ít nhất (210 nút) lại có Max Frontier lớn nhất (67 so với 8 của BFS). Như vậy số nút mở rộng thấp không nhất thiết kéo Max Frontier xuống, và frontier còn phụ thuộc bản đồ, cấu trúc nhánh, heuristic và cách cài đặt.

Câu 7. Tại sao DFS có thể cho đường đi rất dài hoặc gặp vấn đề trong môi trường có chu trình?

DFS ưu tiên đi sâu một nhánh trước khi quay lại nhánh khác; nếu nhánh được xét trước vòng vèo nhưng vẫn tới đích, DFS trả về lời giải dài hơn tối ưu vì không xét các đường ngắn trước. Trong thực nghiệm, DFS cho cost 29 và 152 trên small và medium trong khi tối ưu là 19 và 68. Trong môi trường có chu trình, DFS không kiểm tra chu trình có thể quay lại trạng thái cũ liên tục và không kết thúc. Cài đặt của nhóm dùng is_cycle() đối chiếu với các tổ tiên nên một nhánh không lặp trạng thái, nhưng các nhánh khác nhau vẫn có thể đến cùng một ô; do đó cơ chế này hạn chế chu trình chứ không loại hết đường đi dư thừa (DFS không kết thúc được trên open_maze).

Câu 8. Tại sao thứ tự sinh trạng thái con ảnh hưởng mạnh đến DFS nhưng ít ảnh hưởng hơn đến BFS?

Thứ tự sinh quyết định thứ tự nút vào Queue/Stack. Với DFS dùng stack, nút đẩy vào sau cùng được lấy ra trước nên thứ tự duyệt thực tế ngược với thứ tự sinh, và thứ tự này quyết định nhánh nào được đào sâu trước – ảnh hưởng đến cả độ dài đường đi lẫn số nút mở rộng. BFS vẫn tìm đường ngắn nhất khi mọi bước cùng chi phí, bất kể thứ tự; thứ tự chỉ chọn đường nào trong các đường cùng độ dài ngắn nhất được trả về. Nhóm kiểm chứng bằng cách đổi thứ tự hành động:

Mê cung	Thứ tự	BFS: cost / mở rộng	DFS: cost / mở rộng
small	N,E,S,W	19 / 90	29 / 39
small	S,E,N,W	19 / 90	49 / 78
small	W,S,E,N	19 / 90	37 / 93
medium	N,E,S,W	68 / 267	152 / 153
medium	S,E,N,W	68 / 267	174 / 176
medium	W,S,E,N	68 / 266	244 / 267
large	N,E,S,W	210 / 618	210 / 337
large	S,E,N,W	210 / 616	210 / 406
large	W,S,E,N	210 / 617	210 / 519

Bảng 13. Ảnh hưởng của thứ tự sinh hành động lên BFS và DFS.

Path cost của BFS không đổi (19, 68, 210) và số nút mở rộng chỉ lệch 0–2 nút, trong khi DFS thay đổi mạnh (small: 29 → 49; medium: 152 → 244; large: 337 → 519 nút mở rộng).

Câu 9. So sánh GBFS và A*: thuật toán nào dùng g(n), h(n), f(n)? Ảnh hưởng đến tính tối ưu?

GBFS chỉ dùng h(n) để chọn nút (f(n) = h(n)), tập trung về phía đích nên thường nhanh nhưng không bảo đảm tối ưu; A* dùng f(n) = g(n) + h(n), kết hợp chi phí đã đi với ước lượng còn lại nên tối ưu khi h admissible (và consistent nếu dùng reached). [TV3 bổ sung số liệu thực nghiệm GBFS so với A* trên các mê cung để minh họa.]

Câu 10. Manhattan và Euclidean khác nhau ra sao trong lưới chỉ cho phép đi lên/xuống/trái/phải?

Manhattan (chuẩn L_1) đúng bằng chi phí đường đi ngắn nhất trong lưới 4 hướng không vật cản (h = h*), nên là heuristic hoàn hảo, admissible và consistent. Euclidean (chuẩn L_2) luôn ≤ Manhattan nên cũng admissible nhưng đánh giá thấp chi phí thật (đường chéo không tồn tại) và là heuristic yếu hơn, nên A* thường mở rộng nhiều nút

hơn. Số liệu của nhóm khi w = 1: Expanded của Manhattan / Euclidean là 53 / 56 (small), 221 / 226 (medium), 549 / 557 (large); trên L_maze, A* gặp 66 ô với Manhattan so với 87 ô với Euclidean. Khi w lớn, Euclidean còn có thể lệch mạnh (medium, w = 10: Cost 102 so với 74 của Manhattan).

Câu 11. Weighted A* thay đổi điều gì khi tăng trọng số heuristic?

Chỉ h(n) được nhân trọng số: f(n) = g(n) + w·h(n). Khi w tăng, thuật toán thiên về trạng thái ước lượng gần Goal hơn, làm đổi thứ tự lấy nút khỏi frontier; có thể giảm số nút mở rộng nhưng không bảo đảm tối ưu khi w > 1. Small/Manhattan và Medium/Manhattan cho thấy giảm Expanded kèm tăng Cost; Medium/Euclidean ở w = 10 cho thấy cả Expanded và Cost đều tăng; Large cho thấy Expanded giảm nhưng Max Frontier tăng. Hiệu quả phụ thuộc cấu trúc mê cung và độ phù hợp của h(n).

Câu 12. Cycle checking làm giảm lặp trạng thái nhưng có thể làm tăng chi phí quản lý dữ liệu như thế nào?

Mỗi lần kiểm tra chu trình đều tốn thời gian truy vấn và/hoặc bộ nhớ lưu trạng thái. Kiểm tra theo đường đi hiện tại (is_cycle()) lần theo parent nên mất O(d) mỗi phép kiểm tra (d là độ sâu hiện tại), không cần reached toàn cục nhưng vẫn phải giữ chuỗi nút cha. Nếu dùng hash set reached, mỗi truy vấn trung bình O(1), đổi lại bộ nhớ tăng theo số trạng thái đã ghi nhận. Ngoài ra, quản lý đồng thời frontier, reached và cập nhật trạng thái trùng lặp (như phương án 3 ở Bài 7: xóa mục cũ rồi push lại lên đầu stack) làm tăng chi phí lập trình và duy trì dữ liệu. Vì vậy cycle checking là sự đánh đổi giữa thời gian kiểm tra, bộ nhớ và lượng tìm kiếm lặp có thể tránh được.

3. NHẬN XÉT VÀ KẾT LUẬN

Tính đầy đủ và tối ưu. BFS đầy đủ và tối ưu khi mọi bước có cùng chi phí (cost 19, 68, 210). DFS đầy đủ trong không gian hữu hạn nếu kiểm tra chu trình đúng nhưng không tối ưu (29 và 152 so với 19 và 68) và không kết thúc được trên mê cung mở khi chỉ kiểm tra tổ tiên. IDS đầy đủ và tối ưu như BFS. A* với h admissible và w = 1 tối ưu (53/221/549 nút mở rộng, cost bằng BFS); Weighted A* với w > 1 đánh đổi tối ưu lấy số nút mở rộng ít hơn, nhưng không phải lúc nào cũng giảm (Euclidean, medium, w = 10).

Thời gian và bộ nhớ. DFS mở rộng ít nút nhất nhưng đường dài và max frontier có thể lớn (38 trên large); BFS tốn bộ nhớ O(n) cho reached nhưng frontier nhỏ trên mê cung dạng hành lang; IDS giữ bộ nhớ nhỏ nhưng phải mở rộng lặp (60,179 nút trên large, chậm hơn BFS khoảng 200 lần). A* mở rộng ít nút hơn BFS với heuristic phù hợp.

Heuristic. Manhattan phù hợp hơn Euclidean trong lưới 4 hướng. Heuristic Obstacle-Aware (khoảng cách thật) làm A* chỉ mở rộng đúng các nút trên đường tối ưu nhưng cần BFS tiền xử lý và có Max Frontier lớn hơn trong thí nghiệm. Tăng w là một sự đánh đổi chứ không phải cải tiến mặc định.

Cycle checking. Chống lặp trên đường đi chưa đủ ở không gian mở; cần quản lý frontier (phương án 3) hoặc dùng reached, với chi phí thời gian/bộ nhớ tương ứng.

Hạn chế của báo cáo. Phần GBFS/A* của TV3 và Câu 9 chưa có số liệu thực nghiệm [chờ TV3]; thời gian chạy là một lần đo nên chỉ mang tính tham khảo; các kết luận về bộ nhớ chỉ dựa trên max frontier chứ chưa đo peak memory.

4. TÀI LIỆU SỬ DỤNG TRONG BÀI THỰC HÀNH

- 01_trees.ipynb - Lớp Node, parent, action, cost và truy hồi đường đi.
- 02_Maze_Example.ipynb, 03_Maze_Example_Animation.ipynb - So sánh và minh họa các thuật toán.
- 04_Maze_BFS_vs_A_Star.ipynb - BFS và A* với Euclidean/Manhattan.
- 05_Maze_Example-Animation_Cycle_checking.ipynb - Ba chiến lược cycle checking cho DFS.
- 06_Explore_heuristics.ipynb - Giá trị heuristic trong mê cung rỗng.
- Ex1_Maze.ipynb, Ex2_Maze_simplified.ipynb, Ex3_Show_all_mazes.ipynb - Bài tập thực hành, bản rút gọn và hiển thị mê cung.

5. DANH MỤC TÀI LIỆU THAM KHẢO

[1] S. Russell and P. Norvig, "Artificial Intelligence: A Modern Approach," 4th ed., Pearson, 2021, ch. 3.

[2] M. Hahsler, "Introduction to Artificial Intelligence - Course Repository," [Trực tuyến]. Available: https://github.com/mhahsler/Introduction_to_Artificial_Intelligence.

[3] A. Patel, "Heuristics," Stanford Theory Group (Amit's A* Pages). [Trực tuyến]. Available: http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html.

PHỤ LỤC A. KẾT QUẢ ĐẦY ĐỦ CỦA WEIGHTED A* (54 LƯỢT CHẠY)

Mê cung	Heuristic	w	Path Cost	Expanded	Max Depth	Max Frontier	Search (ms)	Tiền xử lý (ms)
Small	Manhattan	1	19	53	19	8	0.3299	0
Small	Manhattan	1.2	19	50	19	8	0.2396	0
Small	Manhattan	1.5	19	53	19	7	0.3021	0
Small	Manhattan	2	19	55	19	6	0.2591	0
Small	Manhattan	5	29	41	29	5	0.1908	0
Small	Manhattan	10	29	39	29	5	0.2244	0
Small	Euclidean	1	19	56	19	7	0.1975	0
Small	Euclidean	1.2	19	55	19	7	0.1863	0
Small	Euclidean	1.5	19	56	19	7	0.1977	0
Small	Euclidean	2	19	58	19	5	0.1949	0
Small	Euclidean	5	19	61	19	5	0.2084	0
Small	Euclidean	10	29	54	29	4	0.1764	0
Small	Obstacle-Aware	1	19	19	19	7	0.0790	0.4267
Small	Obstacle-Aware	1.2	19	19	19	7	0.0673	0.4267
Small	Obstacle-Aware	1.5	19	19	19	7	0.0662	0.4267
Small	Obstacle-Aware	2	19	19	19	7	0.0655	0.4267
Small	Obstacle-Aware	5	19	19	19	7	0.0653	0.4267
Small	Obstacle-Aware	10	19	19	19	7	0.0741	0.4267
Medium	Manhattan	1	68	221	68	8	1.2673	0
Medium	Manhattan	1.2	68	224	68	8	1.7487	0
Medium	Manhattan	1.5	68	229	68	7	1.1492	0
Medium	Manhattan	2	68	228	72	7	1.0488	0
Medium	Manhattan	5	74	78	74	4	0.3646	0
Medium	Manhattan	10	74	78	74	4	0.4118	0
Medium	Euclidean	1	68	226	68	8	1.2321	0
Medium	Euclidean	1.2	68	226	68	8	1.1552	0
Medium	Euclidean	1.5	68	228	68	8	1.2691	0
Medium	Euclidean	2	68	229	71	8	1.1585	0
Medium	Euclidean	5	68	227	79	9	0.8735	0
Medium	Euclidean	10	102	280	128	10	1.0698	0
Medium	Obstacle-Aware	1	68	68	68	7	0.2478	0.7662
Medium	Obstacle-Aware	1.2	68	68	68	7	0.2261	0.7662
Medium	Obstacle-Aware	1.5	68	68	68	7	0.2389	0.7662
Medium	Obstacle-Aware	2	68	68	68	7	0.2313	0.7662

Mê cung	Heuristic	w	Path Cost	Expanded	Max Depth	Max Frontier	Search (ms)	Tiền xử lý (ms)
Medium	Obstacle-Aware	5	68	68	68	7	0.2304	0.7662
Medium	Obstacle-Aware	10	68	68	68	7	0.2251	0.7662
Large	Manhattan	1	210	549	210	12	3.2585	0
Large	Manhattan	1.2	210	536	210	13	2.5590	0
Large	Manhattan	1.5	210	522	210	14	2.2640	0
Large	Manhattan	2	210	510	210	16	2.1662	0
Large	Manhattan	5	210	495	210	20	2.6153	0
Large	Manhattan	10	210	475	210	21	2.2863	0
Large	Euclidean	1	210	557	210	10	2.8383	0
Large	Euclidean	1.2	210	544	210	12	1.7783	0
Large	Euclidean	1.5	210	520	210	13	1.5933	0
Large	Euclidean	2	210	514	210	14	1.5970	0
Large	Euclidean	5	210	508	210	15	1.5704	0
Large	Euclidean	10	210	488	210	18	1.5104	0
Large	Obstacle-Aware	1	210	210	210	67	0.9101	1.7026
Large	Obstacle-Aware	1.2	210	210	210	67	0.9057	1.7026
Large	Obstacle-Aware	1.5	210	210	210	67	0.7645	1.7026
Large	Obstacle-Aware	2	210	210	210	67	0.7149	1.7026
Large	Obstacle-Aware	5	210	210	210	67	0.7132	1.7026
Large	Obstacle-Aware	10	210	210	210	67	0.7195	1.7026

Bảng 14. Toàn bộ 54 lượt chạy Weighted A của TV4. Preprocessing tính một lần cho mỗi mê cung và tái sử dụng cho cả sáu mức w.*